

Software Testing Assignment

Module-1(Fundamental)



What is SDLC ?

--> SDLC is a structure imposed on the development of a software product that defines the process for planning, implementation, testing, documentation, deployment, & ongoing maintenance & support.

- * **6 stages of SDLC:**

- Requirement Gathering
- Analysis
- Designing
- Implementation
- Testing
- Maintenance

I. Requirement Gathering:

- * You have to collect requirements from clients.
- * **Types of requirements:**
 - i. Functional Requirements
 - ii. Non-Functional Requirements
- * Written format, diagram, Table

II. Analysis:

- * The analysis phase defines the requirements of the system, independent of how these requirements will be accomplished.
- * This analysis represents that "what" phase.

III. Designing:

- * According to requirements designer will design this.

IV. Implementation:

- * Programmer will develop this.

V. Testing:

- * Product Quality
- * For Eg: (i) Regression Testing
 - (ii) Internal Testing
 - (iii) Unit Testing
 - (iv) Application Testing
 - (v) Stress Testing

VI. Maintenance:

- * **Corrective Maintenance:** Correct
- * **Adaptive Maintenance:** When you develop it on 2nd platform
- * **Perfective Maintenance:** When you want to make it perfect



What is software testing?

--> Testing is process used to identify correctness, completeness & quality of developed computer software.

--> Testing is the process of evaluating a system or its components to determine whether it satisfies the specified requirements.

--> Using Testing you can evaluate & check there is any defect or not.

- * **Types of Testing:**
 - i. Manual Testing: Web Testing
 - App Testing
 - Desktop Software Testing

ii. Automation Testing: Automation testing is using software tools to run test cases automatically, check results and compare them to expected outcomes without human effort.

What is agile methodology?

--> Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.

--> Agile methods break the product into small incremental builds. These builds are provided in iterations.

i. Pros:

- * Is a very realistic approach to software development promotes teamwork and cross training.
- * Suitable for fixed or changing requirements Delivers early partial working solutions.
- * Enables concurrent development and delivery within on overall.
- * Planned context.
- * Little or no planning required and delivery Easy to manage.
- * Gives flexibility to developers.

ii. Cons:

- * Not suitable for handling complex dependencies.
- * More risk of sustainability, maintainability and extensibility.
- * An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- * There is very high individual dependency, since there is minimum documentation generated.

What is SRS ?

--> SRS stands for Software Requirements Specification is a formal technical document that explicitly defines the functional, non-functional & behavioral

requirements of a software application.

- * **Key Sections Utilized by Testers:**

- * Functional Requirements: System behaviors, inputs processing steps and specific feature outputs.
- * To write positive and negative functional test cases.
- * Non-Functional Requirements: Performance criteria, security benchmarks, scalability and usability standards.
- * To design load, stress, security and accessibility test suits.
- * Business Rules & Constraints: Legal regulations data validation rules and operating system dependencies.
- * To construct boundaries and edge case testing matrices.



What is oops ? Write basic Concepts of OOP .

--> OOP it stands for object-oriented Programming.

--> An object-based programming language is one which easily supports object-orientation.

--> Smalltalk: 1972-1980

--> Founder of Alan kay

--> C++: 1986, Bjarne Stroustrup

--> java(oak): 1992 (smalltalk + C++)

- * **Features of OOP:**

i. Object: Any Entity which has own state and Behaviour.

For Ex: Pen, Paper, Chair etc

ii. Class: Collection of objects

For Ex: Human Body

iii. Abstraction: Hiding eternal details and showing functionalities

For Ex: FB login page

iv. Encapsulation: Wrapping up of data or binding of data.

For Ex: Capsule

v. Inheritance: When you object acquire all the properties & behaviour of parent class.

For Ex: Father-Son

vi. Polymorphism: Many ways to perform anything.

For Ex: Road Ways

Method Overloading

Method Overriding

What is object ?

--> An "object" is anything to which a concept applies.

--> An object represents an individual, identifiable item, unit, or entity, either real or abstract, with a well-defined role in the problem domain.

What is class ?

--> When you define a class, you define a blueprint for an object.

--> A class represents an abstraction of the object and abstracts the properties and behaviour of that object.

--> An object is a particular instance of a class which has actual existence & there can be many objects for a class.

- * Making Classes: Creating extending or reusing abstract data types.
- * Making Objects interact: Creating objects from abstract data types and defining their relationships.

What is encapsulation ?

--> encapsulation is the practice of including in an object everything it needs hidden from other objects. The internal state is usually not accessible by other objects.

--> Encapsulation enables data hiding, hiding irrelevant information from the users of a class & exposing the relevant details required by the user.

What is inheritance ?

--> Inheritance means that the class inherits the characteristics of another class. This is also called a "is a" relationship.

--> In a class context, inheritance is referred to as implementation inheritance, and in an interface context, it is also referred to as interface inheritance.

i. Grandparent

ii. Parent

iii. Child

What is polymorphism ?

--> Polymorphism means "having many forms".

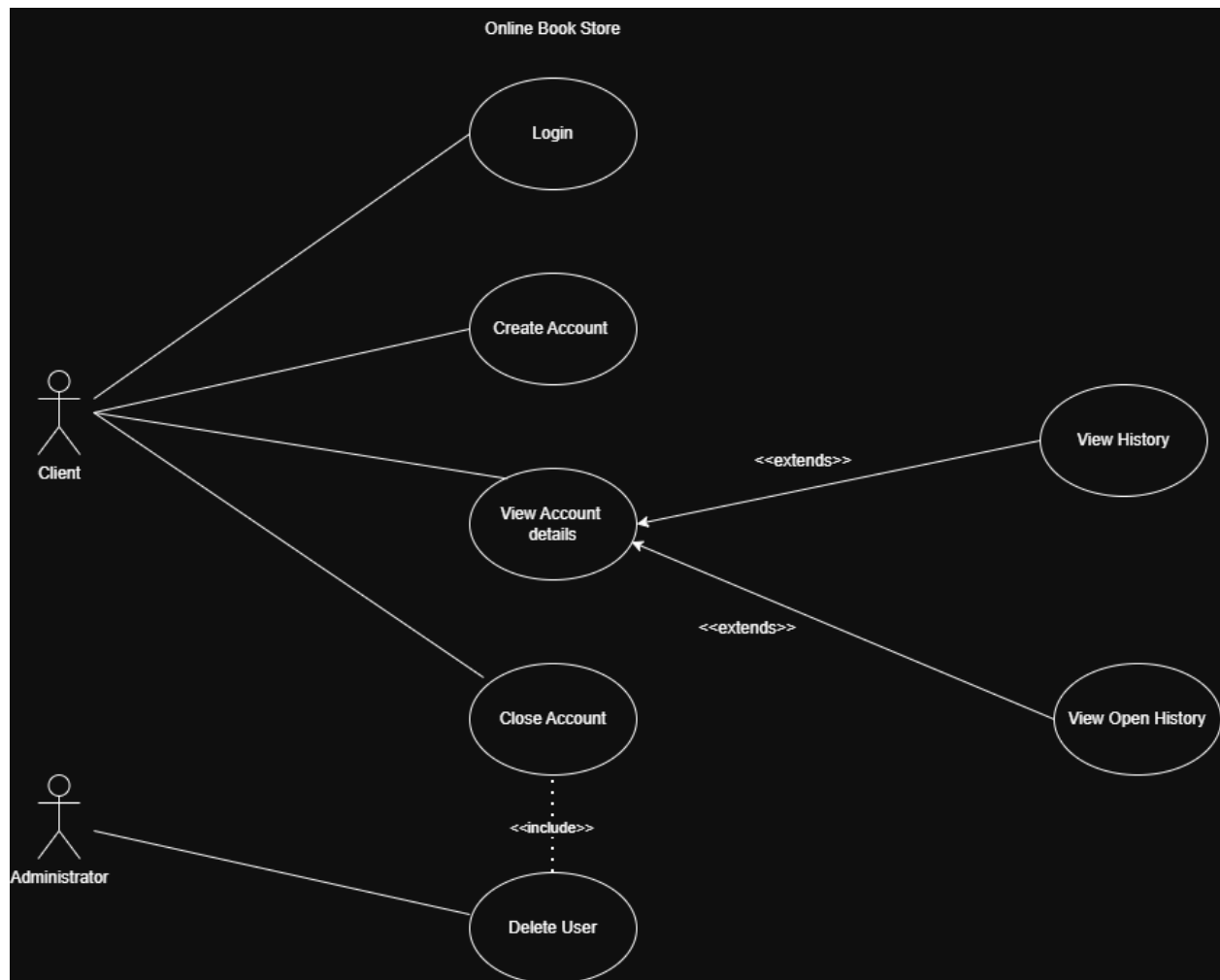
--> It allows different objects to respond to the same message in different ways, the responses specific to the types of the object.

--> The ability to change form is known as polymorphism.

i. Overloading: The same method name (method overloading) or operator symbol (operator overloading) can be used in different contexts.

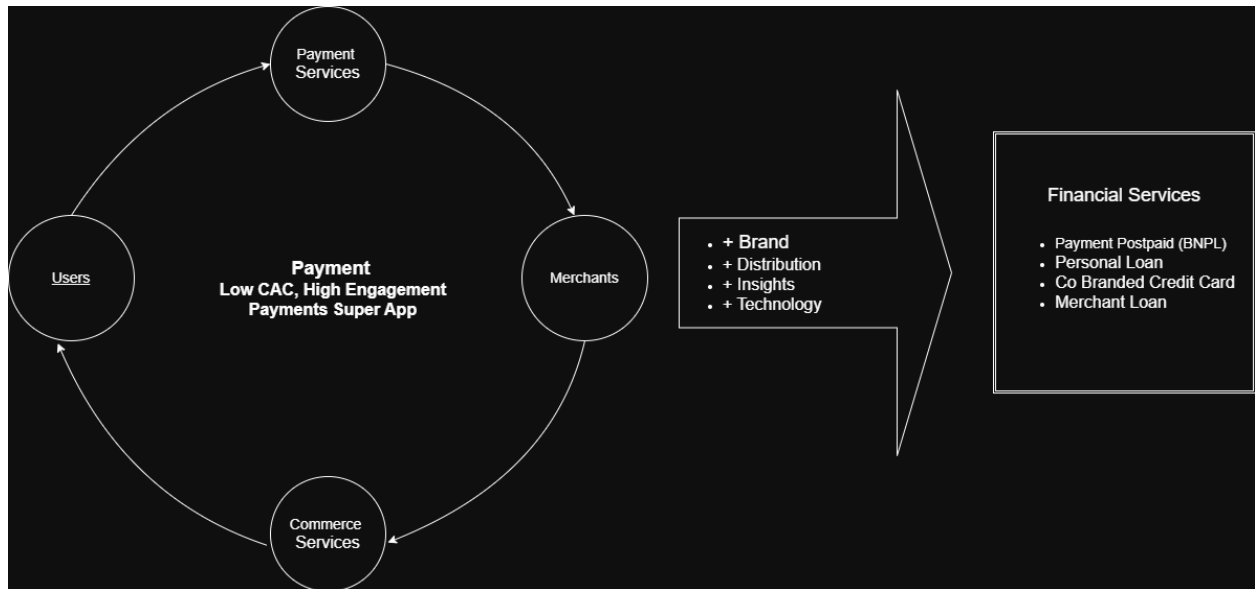
Draw Usecase on Online book shopping ?

-->



 Draw Usecase on online bill payment system (paytm) ?

-->



🕒 Write SDLC phases with basic introduction ?

--> There are 6 SDLC Phases

- * Requirement Gathering
- * Analysis
- * Design
- * Implementation
- * Testing
- * Maintenance

i. Requirement Gathering: Although requirements may be documented in written form, they may be incomplete, unambiguous, or even incorrect.

--> Requirements definitions usually consist of natural language supplemented by diagrams and tables

* Types of Requirements:

a. Functional Requirements: Describe system services or functions.

--> Compute sales tax on a purchase
Update the database on the server.

b. Non-Functional Requirements: Non-Functional requirements may be more critical than functional requirements.

--> If these are not met, the system is useless!

ii. Analysis Phase: The analysis phase defines the requirements of the system, independent of how these requirements will be accomplished.

--> This analysis represents the "What" phase.

iii. Design Phase: Design Architecture Document implementation Plan

--> Critical Priority Analysis Performance Analysis Test Plan

--> The architecture team also converts the typical scenarios into a test plan.

iv. Implementation Phase: In the implementation phase, the team builds the components either from scratch or any composition.

--> The implementation phase deals with issues of quality, performance, baselines, libraries, and debugging.

v. Testing Phase: Simply stated, quality is very important. Many companies have not quality is important and deliver more claimed functionality but at a lower quality level.

--> A customer satisfied with the quality of a product will remain loyal and wait for new functionality in the next version.

vi. Maintenance Phase: Maintenance is the process of changing a system after it has been deployed.

- * Corrective Maintenance: Identifying and repairing defects
- * Adaptive Maintenance: Adapting the existing solution to the new platforms
- * Perfective Maintenance: Implementing the new requirements

Explain Phases of the waterfall model

--> There are 6 phases in waterfall Model.

i. Requirements Analysis: Testers analyze the requirements Specification document.

--> Goal is to find ambiguities (things that are unclear, vague) early.

--> Testing team creates the **Test Plan**.

ii. System Design: Testers review design documents for testability.

--> Testing team designs the **Test Strategy**.

--> Integration test scenarios are planned here.

iii. Implementation Design (Coding): Developers perform **Unit Testing** on modules.

--> Testers execute write detailed **Test Cases**.

--> Test environments and data are prepared.

iv. Integration & Testing: Entire system is assembled and tested.

--> Testers execute functional and regression tests.

--> **Users Acceptance Testing (UAT)** is conducted.

v. Deployment: Software is released to production environments.

--> Testers perform final production sanity checks.

--> Ensures deployment did not introduce defects.

vi. Maintenance: Software is updated for production bugs.

--> System is modified to enhance performance or features.

--> Testing perform **Regression Testing** on all new patches (small, quick software updates used to fix bugs or security flaws in an existing program).



Write phases of spiral model.

--> There are 4 phases in spiral model.

i. Objectives & Requirements Evaluation: Testers review requirements for the current spiral model.

--> Testing Team identifies target success criteria and constraints.

--> High-Level **Test Requirements** and boundaries are defined.

ii. Risk-Based Testing & Prototyping: Testers perform Risk Analysis to catch critical flaws early.

--> Prototypes are verified to mitigate (to bring under control) high priority technical risks.

--> **Risk-based Testing (RBT)** strategies are formally established here.

iii. Test Execution & Product verification: Developers write the iteration code and perform **Unit Testing**.

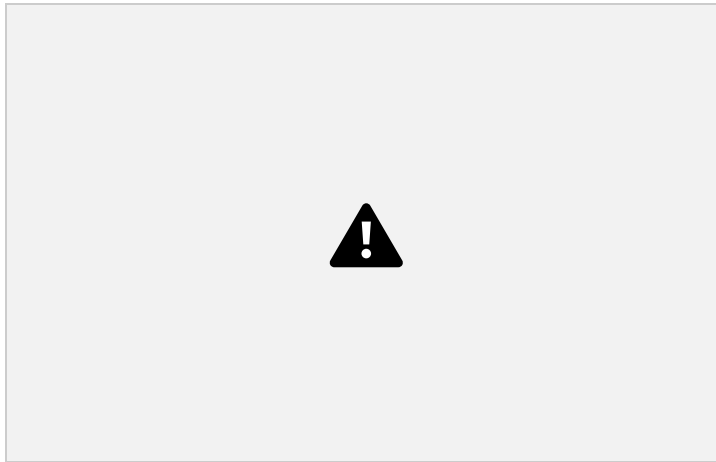
--> Testers execute **Integration Testing** and **System Testing** cycles.

--> Bugs are logged, tracked, fixed inside the current loop.

iv. Review & Next-Cycle Test Planning: Stakeholders perform **User Acceptance Testing (UAT)** on the iteration.

--> Testing team evaluates test coverage metrics and remaining defects.

--> Testers update the **Test Plan** to prepare for the next spiral.



 **Explain working methodology of agile model and also write pros and cons.**

--> Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.

--> Each Iteration typically lasts from about one to another.

* **What is Agile?**

--> Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to test suit the project requirements. In agile the tasks are divided to time boxes (small time frames) to deliver specific features for a release.

* Pros:

--> Is a very realistic approach to software development promotes teamwork and cross training.

--> Functionality can be developed rapidly and demonstrated. Resources requirements are minimum.

--> Planned context.

--> Gives flexibility to developers.

* Cons:

--> Not suitable for handling complex dependencies.

--> More risks of sustainability, maintainability and extensibility.

--> An overall plan, an agile leader and agile PM practice is a must without which it will work.

--> Transfer of technology to new team members may be quite challenging due to lack of documentation.



Write agile manifesto principles.

--> However, the software testing community adapts the 12 agile principles to focus on quality, continuous feedback & collaboration.

* **Continuous Value & Delivery:**

i. Satisfy the customer through early delivery:

--> Test the customer from day one to ensure high-quality software reaches the user quickly.

ii. Welcome changing requirements:

--> Update test cases and automation scripts flexibly to support changing business goals, even late in development.

iii. Deliver working software frequently:

--> Execute fast, reliable regression tests to support frequent software releases.

* **Collaboration & Culture:**

i. Work together daily:

--> Developers, testers & business stakeholders must communicate constantly to prevent bugs before writing code.

ii. Build projects around motivated individuals:

--> Trust testing teams with the right tools and authority to take ownership of product equality.

iii. Use face-to-face conversation:

--> Discuss defects and quality risks directly with developers instead of relying only on written bug reports.

* **Progress & Excellence:**

i. Working software is the primary measure of progress:

--> Test coverage alone does not mean success; a stable, bug-free, working product does.

ii. Maintain a sustainable pace:

--> Keep test automation and testing efforts steady so the team does not burn out before releases.

iii. Continuous attention to technical excellence:

--> Focus on strong test architecture, clean automation code, and robust non-functional testing.

* **Simplicity & Feedback:**

i. Simplicity is essential:

--> Write simple, clear test cases and automation only what delivers high value to save time.

ii. Self-organizing teams:

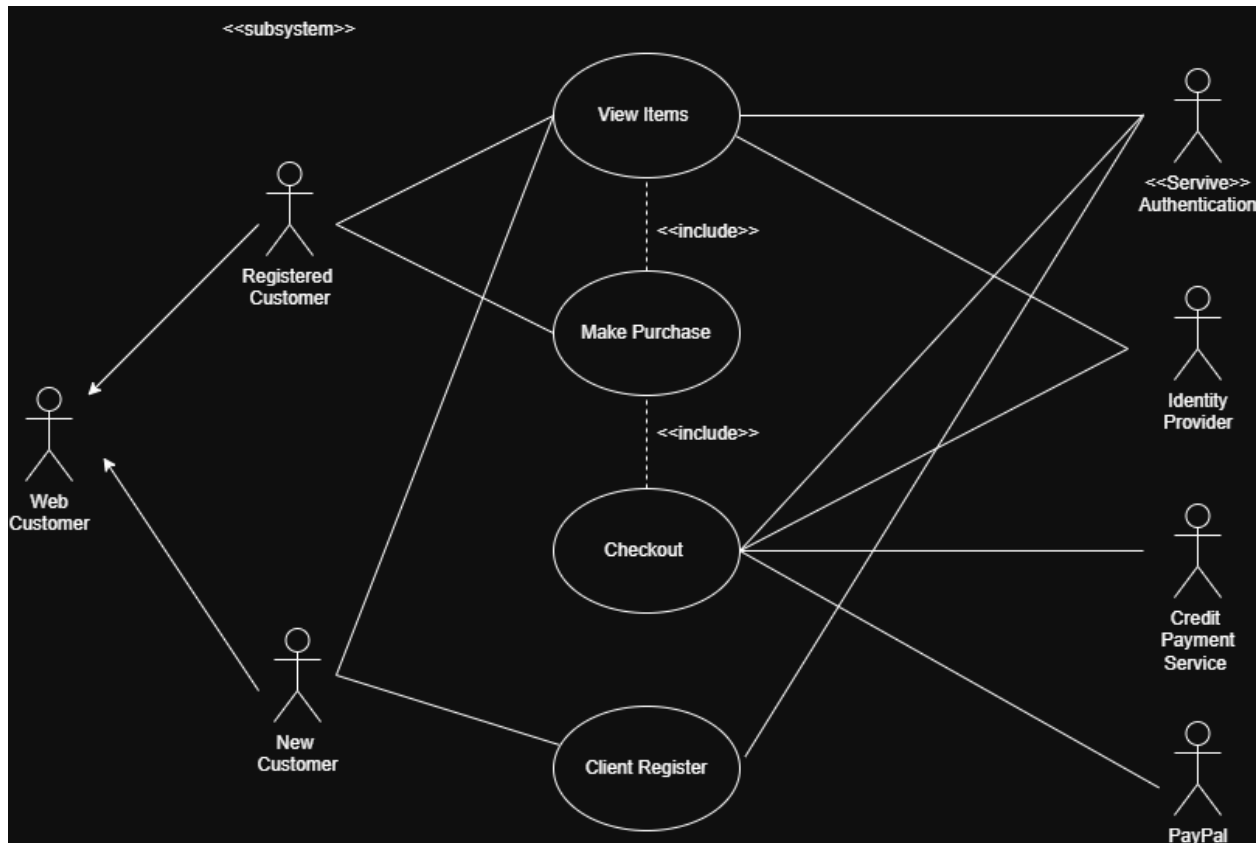
--> Empower the whole team to share testing responsibilities and define the best quality strategies together.

iii. Regularly reflect and adjust:

--> Review testing processes during retrospectives (regular meetings held by a team at the end of a project cycle, sprint, or milestone to look back at their performance and find ways to improve) efficiency and reduce future defects.

 Draw usecase on Online shopping product using COD.

-->



 Draw usecase on Online shopping product using payment gateway.

-->

